axiateam
RDT company

Módulo 9

Genéricos

Curso: TypeScript

90 min

Objetivos de Aprendizaje

- Comprender el concepto de **genéricos**
- Crear **funciones genéricas** reutilizables
- Implementar **clases e interfaces** genéricas
- Aplicar **restricciones** a genéricos
- Utilizar genéricos **built-in** de TypeScript

1. Contexto

Reutilización de Código

El Problema

Sin genéricos, necesitamos repetir código:

```
// Versión para strings
function identityString(value: string): string {
  return value;
}

// Versión para números
function identityNumber(value: number): number {
  return value;
}

// Versión para booleanos
function identityBoolean(value: boolean): boolean {
  return value;
}
```

La Solución: Genéricos

```
function identity<T>(value: T): T {
  return value;
}

identity<string>("hello");
identity<number>(42);
identity(true); // TypeScript infiere el tipo
```

2. Funciones Genéricas

Generic Functions

Las funciones genéricas trabajan con cualquier tipo manteniendo la seguridad tipográfica.

```
// Función genérica básica
function firstElement<T>(arr: T[]): T | undefined {
  return arr[0];
}

// Múltiples tipos genéricos
function pair<K, V>(key: K, value: V): [K, V] {
  return [key, value];
}

// Restricciones con extends
function getProperty<T, K extends keyof T>(obj: T, key: K): T[K] {
  return obj[key];
}
```

Lab: Utilidades Genéricas para Tareas

Objetivo: Crear funciones genéricas reutilizables.

1. Implementa `wrapInArray`
2. Implementa `mapArray`
3. Implementa `filterByProperty`
4. Implementa `sortByProperty`

Código del Lab

```
function wrapInArray<T>(value: T): T[] {
  return [value];
}

function mapArray<T, U>(arr: T[], fn: (item: T) => U): U[] {
  return arr.map(fn);
}

function filterByProperty<T, K extends keyof T>(
  arr: T[],
  key: K,
  value: T[K]
): T[] {
  return arr.filter(item => item[key] === value);
}

function sortByProperty<T, K extends keyof T>(
  arr: T[],
  key: K
```

3. Clases Genéricas

Generic Classes

Las clases genéricas permiten crear componentes reutilizables que trabajan con cualquier tipo.

```
// Repository genérico
class Repository<T> {
  private items: T[] = [];

  add(item: T): void {
    this.items.push(item);
  }

  getAll(): T[] {
    return [...this.items];
  }

  find(predicate: (item: T) => boolean): T[] {
    return this.items.filter(predicate);
  }
}

const taskRepo = new Repository<Task>();
const userRepo = new Repository<User>();
```

Lab: Repository Genérico para Tareas

Objetivo: Implementar un repositorio genérico.

1. Crea la clase `TaskRepository`
2. Implementa métodos CRUD
3. Añade métodos de búsqueda

Código del Lab

```
// TaskRepository (versión específica)
class TaskRepository {
  private tasks: Task[] = [];

  add(task: Task): void { this.tasks.push(task); }
  getAll(): Task[] { return [...this.tasks]; }
  findById(id: string): Task | undefined {
    return this.tasks.find(t => t.id === id);
  }
  update(task: Task): boolean {
    const index = this.tasks.findIndex(t => t.id === task.id);
    if (index > -1) { this.tasks[index] = task; return true; }
    return false;
  }
  delete(id: string): boolean {
    const index = this.tasks.findIndex(t => t.id === id);
    if (index > -1) { this.tasks.splice(index, 1); return true; }
    return false;
  }
```

4. Restricciones (Constraints)

Constraints con extends

Las restricciones limitan los tipos permitidos:

```
// Requiere que T tenga id
interface Identifiable {
  id: string;
}

function findById<T extends Identifiable>(
  items: T[],
  id: string
): T | undefined {
  return items.find(item => item.id === id);
}

// Múltiples restricciones
function createPair<K extends string, V extends { toString(): string }>(
  key: K,
  value: V
): { key: K; value: string } {
  return { key, value: value.toString() };
}
```

Lab: Sistema de Filtrado Genérico

Objetivo: Crear filtros genéricos con restricciones.

1. Define interfaz base `Filterable`
2. Implementa filtro genérico con `extends`
3. Añade múltiples criterios de filtrado

```
interface Filterable {
  status?: TaskStatus;
  priority?: TaskPriority;
}

class TaskFilter<T extends Filterable> {
  filter(items: T[], criteria: Partial<T>): T[] {
    return items.filter(item => {
      for (const key in criteria) {
        if (criteria[key] !== undefined &&
            item[key] !== criteria[key]) {
          return false;
        }
      }
      return true;
    });
  }
}
```

5. Conditional Types

Conditional Type Basics

Los conditional types permiten crear tipos que dependen de otros tipos:

```
// Sintaxis: T extends U ? X : Y
// Si T es asignable a U, el tipo es X, si no, es Y

type IsString<T> = T extends string ? "yes" : "no";

type A = IsString<string>;  // "yes"
type B = IsString<number>;  // "no"
```

Ejemplo: ElementType

Extraer el tipo de elemento de un array:

```
type ElementType<T> = T extends Array<infer Item> ? Item : never;

type C = ElementType<string[]>;  // string
type D = ElementType<number[]>;  // number
type E = ElementType<boolean>;   // never
```

Keyword: infer

`infer` permite extraer tipos dentro de conditional types:

```
// Extraer tipo de retorno de una función
type ReturnType<T> = T extends (...args: any[]) => infer R ? R : never;

function getTask() {
  return { id: "1", title: "Test" };
}

type TaskReturn = ReturnType<typeof getTask>;
// { id: string; title: string }
```

infer: Extraer Argumentos

```
// Extraer primer argumento de una función
type FirstArg<T> = T extends (first: infer F, ...rest: any[]) => any ? F : never;

function createTask(title: string, priority: number) {}

type TitleArg = FirstArg<typeof createTask>;  // string
```

Distributive Conditional Types

Los conditional types se distribuyen sobre tipos unión:

```
type ToArray<T> = T extends any ? T[] : never;

type StringOrNumberArray = ToArray<string | number>;
// Equivale a: ToArray<string> | ToArray<number>
// Resultado: string[] | number[]
```

Ejemplo: Exclude

Filter con conditional types:

```
type Exclude<T, U> = T extends U ? never : T;

type Status = "pending" | "inProgress" | "completed" | "blocked";
type ActiveStatus = Exclude<Status, "completed" | "blocked">;
// "pending" | "inProgress"
```

Lab: Conditional Types

Objetivo: Crear utility types personalizados usando conditional types.

1. Implementa `UnwrapPromise`
2. Implementa `ExtractArrayElement`
3. Implementa `Nullable`

Código del Lab

```typescript
// 1. UnwrapPromise
type UnwrapPromise<T> = T extends Promise<infer R> ? R : T;
type A = UnwrapPromise<Promise<string>>;  // string
type B = UnwrapPromise<number>;             // number

// 2. ExtractArrayElement
type ExtractArrayElement<T> = T extends Array<infer E> ? E : T;
type C = ExtractArrayElement<string[]>;  // string

// 3. Nullable
type Nullable<T> = T | null | undefined;
type E = Nullable<string>;  // string | null | undefined
```

6. Utility Types

Partial, Required, Readonly

Transforma tipos: opcionales, requeridos, solo lectura.

```
interface User {
  id: string;
  name: string;
  email: string;
}

type UserUpdate = Partial<User>;
type UserComplete = Required<User>;
type UserView = Readonly<User>;
```

Pick, Omit, Record

Selecciona, excluye propiedades o crea tipos con claves específicas.

```
type UserPreview = Pick<User, "id" | "name">;
type UserWithoutEmail = Omit<User, "email">;

type UserRole = "admin" | "user" | "guest";
type RolePermissions = Record<UserRole, string[]>;
```

Exclude, Extract, NonNullable

Filtra tipos de unión o elimina nulos.

```
type Status = "pending" | "inProgress" | "completed" | null;

type ActiveStatus = Exclude<Status, "completed">;
type ProcessingStatus = Extract<Status, "pending" | "inProgress">;
type NonNullStatus = NonNullable<Status>;
```

ReturnType y Parameters

Extrae el tipo de retorno o parámetros de una función.

```
function createTask(title: string): Task {
  return { id: "1", title, status: "pending" };
}

type TaskReturn = ReturnType<typeof createTask>;
type TaskParams = Parameters<typeof createTask>;
```

Lab: Utility Types en la Biblioteca

Objetivo: Aplicar utility types para crear variantes de tipos.

1. Crea `BookUpdate` con `Partial<Book>`
2. Crea `BookPreview` con `Pick`
3. Crea `MemberStats` con `Record`
4. Usa `ReturnType` para obtener tipo de función

Código del Lab

```
type BookUpdate = Partial<Book>;
type BookPreview = Pick<Book, "title" | "author">;

type MemberRole = "standard" | "premium" | "admin";
type MemberStats = Record<MemberRole, { count: number }>;

function getLoanStatus(loan: Loan): "active" | "returned" {
  return loan.returned ? "returned" : "active";
}

type LoanStatus = ReturnType<typeof getLoanStatus>;
```

Resumen

Puntos Clave

- Los **genéricos** permiten crear código reutilizable con tipos variables
- `<T>` define un parámetro de tipo
- **Restricciones** con `extends` limitan los tipos permitidos
- TypeScript infiere el tipo en la mayoría de casos
- **Utility types** como `Partial`, `Required`, `Pick`

Ejercicio Integrador

Sistema de Biblioteca

Caso 2: Biblioteca

Objetivo: Aplicar genéricos al sistema de biblioteca.

1. Crea un repositorio genérico para libros y miembros
2. Implementa filtros genéricos
3. Crea utility types para el dominio

Trabajo en grupos de 2-4 personas.

Tips y Gotchas

✅ Tips

- Genéricos permiten código reusable manteniendo tipos
- Usa `extends` para constraints
- Utility types: `Partial<T>`, `Pick<T,K>`, `Omit<T,K>`

⚠️ Gotchas

- `Array<T>` vs `T[]` son equivalentes
- Constraint `extends` no es herencia
- No abuses de `any` genérico - especifica constraints

Referencias

Recursos Adicionales

- Generics
- Utility Types

Gracias!

Módulo 9 Completado

Siguiente: Módulo 10 - Módulos